

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра информационных технологий

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7
по дисциплине
«Операционные системы»

Выполнил студент группы ПМ23 _____ Р. В. Касьянов

Направление подготовки 01.03.02 Прикладная математика и информатика
Курс 2

Принял,
канд. техн. наук, доцент каф. ИТ _____ А.А. Полупанов

Краснодар
2026

Задание 1.

1. Создайте скрипт приветствие `hello.sh`.

```
ruslan@dc-1:~$ nano hello.sh
```

Рисунок 1 – Создание файла `hello.sh` с помощью редактора `nano`

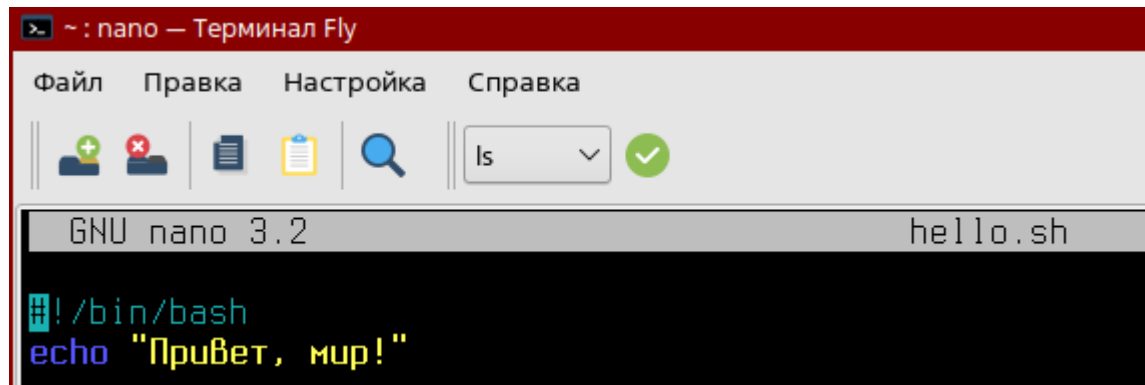


Рисунок 2 – Изменение файла `hello.sh` с помощью редактора `nano`

2. Назначьте скрипт запускаемым и выполните.

```
ruslan@dc-1:~$ chmod u+x hello.sh
ruslan@dc-1:~$ ./hello.sh
Привет, мир!
ruslan@dc-1:~$
```

Рисунок 3 – Установка права на выполнение (`chmod u+x`) и запуск скрипта `./hello.sh`

3. Выполните приветствие через `bash`.

```
ruslan@dc-1:~$ bash hello.sh
Привет, мир!
ruslan@dc-1:~$
```

Рисунок 4 – Запуск скрипта с помощью интерпретатора `bash`

Задание 2.

1. Задайте переменную для цифр, которые будем использовать в простом пароле.

A screenshot of a terminal window showing the nano text editor editing a file named 'alphabet.sh'. The content of the file is: `#!/bin/bash` followed by `digits="0123456789"` on the next line. The status bar at the bottom shows 'GNU nano 3.2' and 'alphabet.sh'.

Рисунок 5 – Фрагмент скрипта `alphabet.sh` с объявлением переменной `digits`

2. Задайте переменную для спец. символов, которые будут в сложном пароле.

```
GNU nano 3.2                                alphabet.sh

#!/bin/bash
digits="0123456789"
special="+-=_~!@#%^"
```

Рисунок 6 – Фрагмент скрипта alphabet.sh с объявлением переменной special

3. Задайте переменную алфавита из символов верхнего регистра A-Z + нижнего a-z + цифр + спец. символы.

```
GNU nano 3.2                                alphabet.sh

#!/bin/bash
digits="0123456789"
special="+-=_~!@#%^"
lower="abcdefghijklmnopqrstuvwxyz"
upper="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
alphabet="$upper$lower$digits$special"
echo "Алфавит: $alphabet"
```

Рисунок 7 – Фрагмент скрипта alphabet.sh с объединением переменных в alphabet

4. Выведите весь алфавит.

```
ruslan@dc-1:~$ nano alphabet.sh
ruslan@dc-1:~$ chmod u+x alphabet.sh
ruslan@dc-1:~$ ./alphabet.sh
Алфавит: ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+-=_~!@#%^
ruslan@dc-1:~$
```

Рисунок 8 – Результат выполнения скрипта alphabet.sh – вывод сгенерированного алфавита

Задание 3.

1. Вычислите в переменную количество символов в алфавите легкого пароля (A-Z + a-z + 0-9).

```
GNU nano 3.2                                easy_password.sh

#!/bin/bash
digits="0123456789"
lower="abcdefghijklmnopqrstuvwxyz"
upper="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
light_alphabet="$upper$lower$digits"
light_len=${#light_alphabet}
```

Рисунок 9 – Скрипт easy_password.sh: вычисление длины легкого алфавита (строка light_len)

2. Вычислите положение случайного символа для легкого пароля.

```
pos=$(( RANDOM % light_len ))
```

Рисунок 10 – Генерация случайной позиции (pos=\$((RANDOM % light_len)))

3. Вырежьте символ из алфавита по случайному положению.

```
char=${light_alphabet:$pos:1}
```

Рисунок 11 – Извлечение символа

4. Приклейте вырезанный символ к простому паролю.

```
password="$password$char"
```

Рисунок 12 – Добавление символа к строке password (password="\${password}\${char}")

5. Повторите действие в цикле, добавив 14 случайных символов.

```
GNU nano 3.2 easy_password.sh

#!/bin/bash
digits="0123456789"
lower="abcdefghijklmnopqrstuvwxyz"
upper="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
light_alphabet="$upper$lower$digits"
light_len=${#light_alphabet}
password=""
for i in {1..14}; do
    pos=$(( RANDOM % light_len ))
    char=${light_alphabet:$pos:1}
    password="$password$char"
done
echo "Легкий пароль: $password"
```

Рисунок 13 – Цикл for от 1 до 14 для накопления пароля

6. Выведите легкий пароль из 14 случайных символов.

```
ruslan@dc-1:~$ chmod u+x easy_password.sh
ruslan@dc-1:~$ ./easy_password.sh
Легкий пароль: 3Koqdu400vk8BA
ruslan@dc-1:~$
```

Рисунок 14 – Результат выполнения скрипта easy_password.sh – сгенерированный пароль

Задание 4.

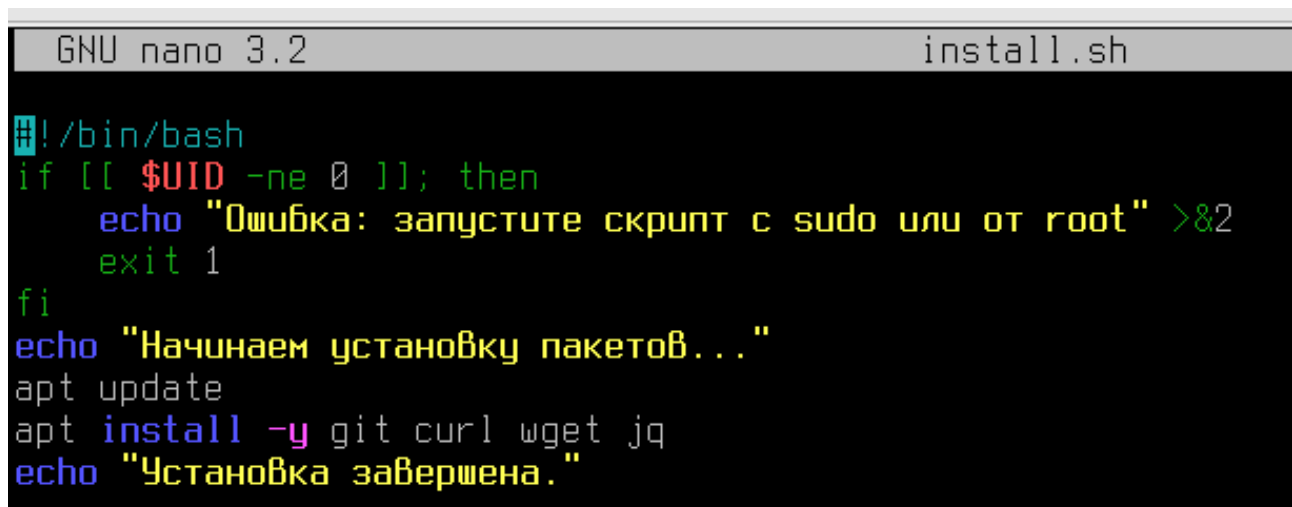
1. Проверьте, является ли текущий пользователь суперпользователем root.

```
GNU nano 3.2 install.sh

#!/bin/bash
if [[ $UID -ne 0 ]]; then
```

Рисунок 15 – Скрипт install.sh: условие if [[\$UID -ne 0]]

2. Если пользователь не является root, то выведите сообщение об ошибке и завершите работу скрипта.

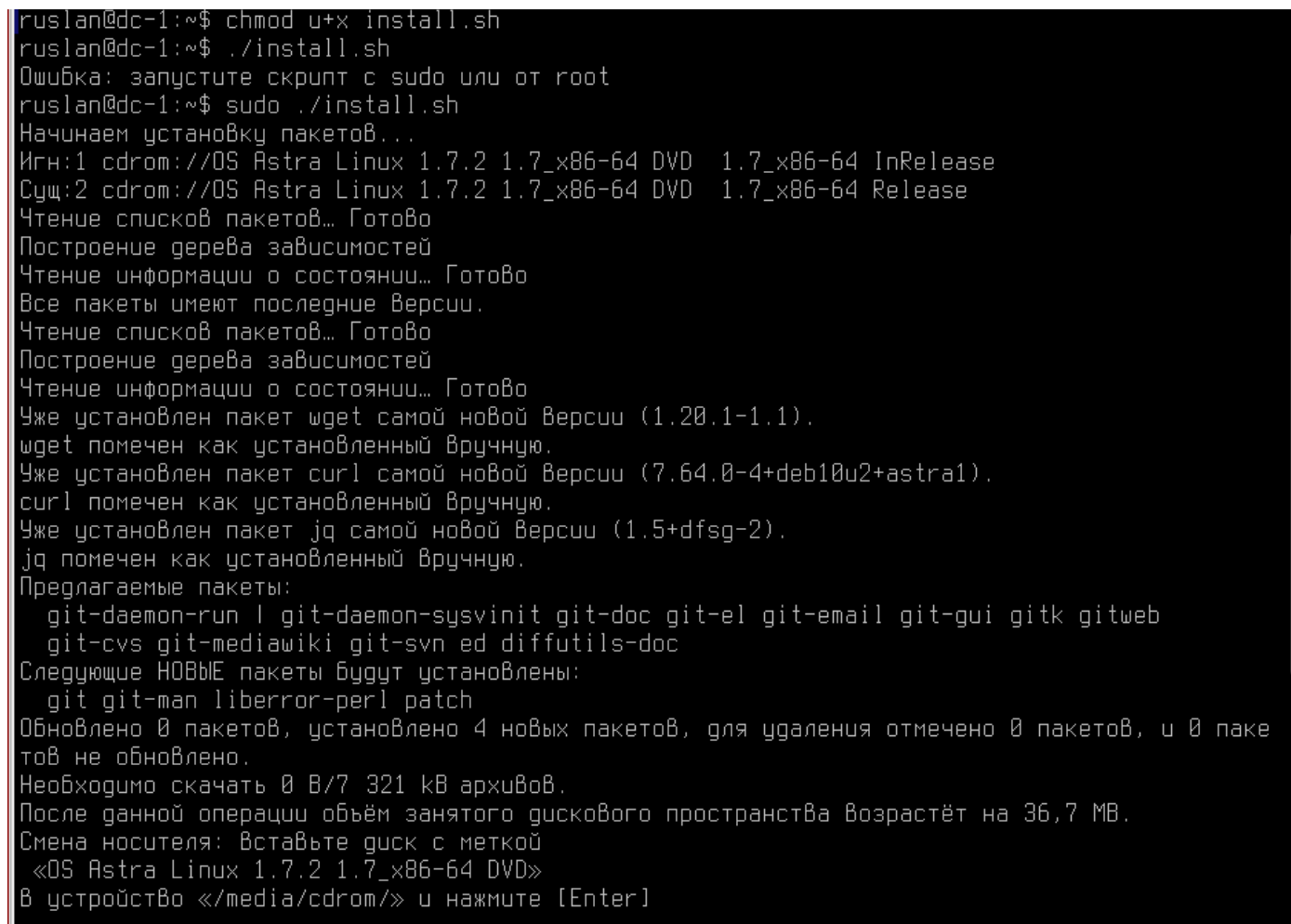


```
GNU nano 3.2                                install.sh

#!/bin/bash
if [[ $UID -ne 0 ]]; then
    echo "Ошибка: запустите скрипт с sudo или от root" >&2
    exit 1
fi
echo "Начинаем установку пакетов..."
apt update
apt install -y git curl wget jq
echo "Установка завершена."
```

Рисунок 16 – Вывод сообщения об ошибке и exit 1

3. Если пользователь root, то установите apt install -y git curl wget jq.

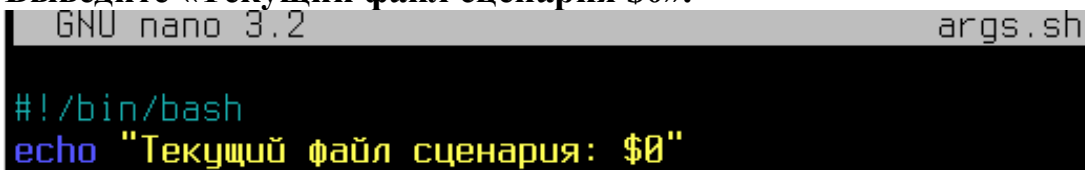


```
ruslan@dc-1:~$ chmod u+x install.sh
ruslan@dc-1:~$ ./install.sh
Ошибка: запустите скрипт с sudo или от root
ruslan@dc-1:~$ sudo ./install.sh
Начинаем установку пакетов...
Игн:1 cdrom://OS Astra Linux 1.7.2 1.7_x86-64 DVD 1.7_x86-64 InRelease
Сущ:2 cdrom://OS Astra Linux 1.7.2 1.7_x86-64 DVD 1.7_x86-64 Release
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Все пакеты имеют последние версии.
Чтение списков пакетов... Готово
Построение дерева зависимостей
Чтение информации о состоянии... Готово
Уже установлен пакет wget самой новой версии (1.20.1-1.1).
wget помечен как установленный вручную.
Уже установлен пакет curl самой новой версии (7.64.0-4+deb10u2+astral).
curl помечен как установленный вручную.
Уже установлен пакет jq самой новой версии (1.5+dfsg-2).
jq помечен как установленный вручную.
Предлагаемые пакеты:
  git-daemon-run | git-daemon-sysvinit git-doc git-el git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn ed diffutils-doc
Следующие NOBbIE пакеты будут установлены:
  git git-man liberror-perl patch
Обновлено 0 пакетов, установлено 4 новых пакетов, для удаления отмечено 0 пакетов, и 0 паке
тов не обновлено.
Необходимо скачать 0 B/7 321 kB архивов.
После данной операции объём занятого дискового пространства возрастёт на 36,7 MB.
Смена носителя: Вставьте диск с меткой
«OS Astra Linux 1.7.2 1.7_x86-64 DVD»
В устройство «/media/cdrom/» и нажмите [Enter]
```

Рисунок 17 – Выполнение скрипта с sudo: процесс установки пакетов

Задание 5.

1. Выведите «Текущий файл сценария \$0».

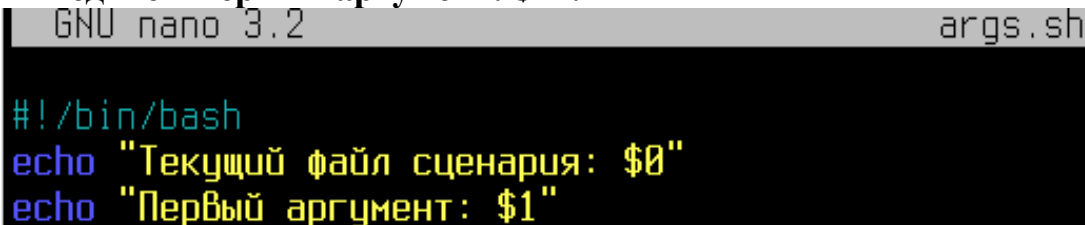


```
GNU nano 3.2 args.sh

#!/bin/bash
echo "Текущий файл сценария: $0"
```

Рисунок 18 – Скрипт args.sh: команда echo с \$0

2. Выведите «Первый аргумент: \$1».

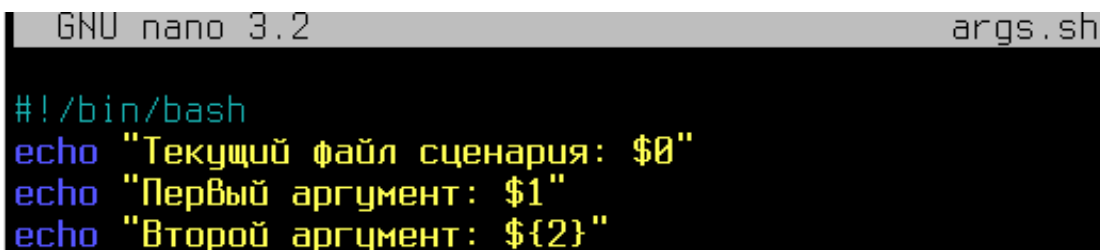


```
GNU nano 3.2 args.sh

#!/bin/bash
echo "Текущий файл сценария: $0"
echo "Первый аргумент: $1"
```

Рисунок 19 – Вывод первого аргумента

3. Выведите «Второй аргумент: \${2}».

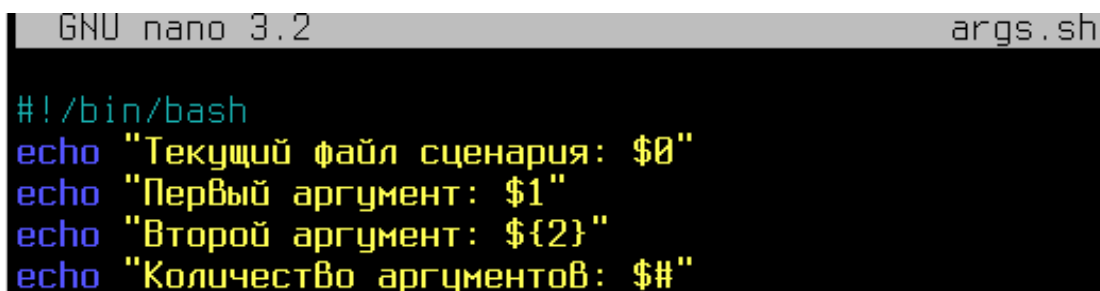


```
GNU nano 3.2 args.sh

#!/bin/bash
echo "Текущий файл сценария: $0"
echo "Первый аргумент: $1"
echo "Второй аргумент: ${2}"
```

Рисунок 20 – Вывод второго аргумента

4. Выведите «Последний аргумент \${!#}».

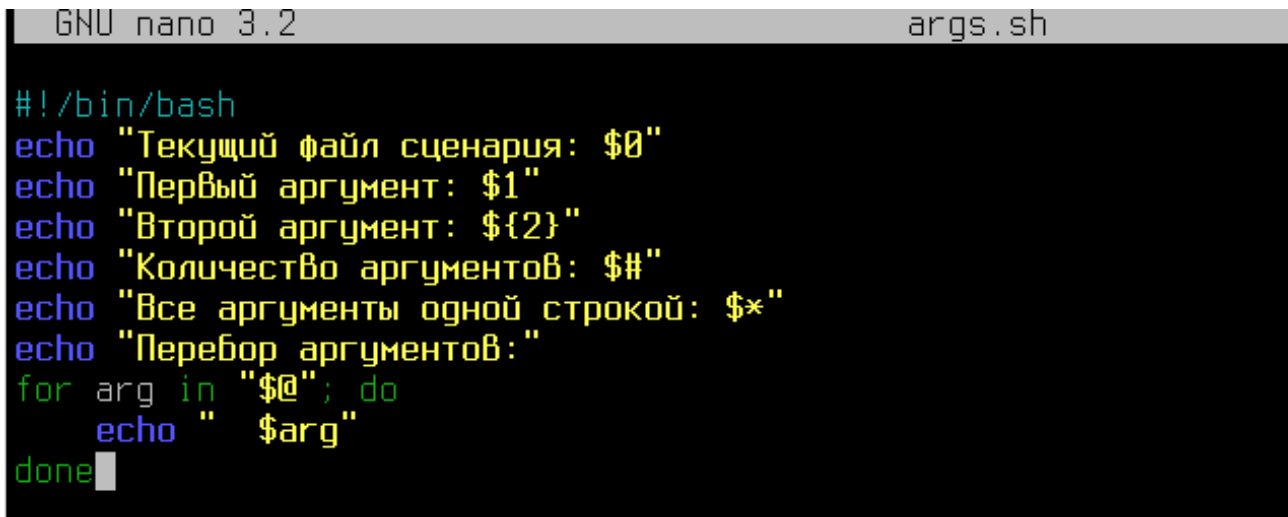


```
GNU nano 3.2 args.sh

#!/bin/bash
echo "Текущий файл сценария: $0"
echo "Первый аргумент: $1"
echo "Второй аргумент: ${2}"
echo "Количество аргументов: $#"
```

Рисунок 21 – В скрипте выводится количество аргументов (\$#), но не последний аргумент; на рисунке видна соответствующая строка

5. Выведите «Все аргументы одной переменной \$*».



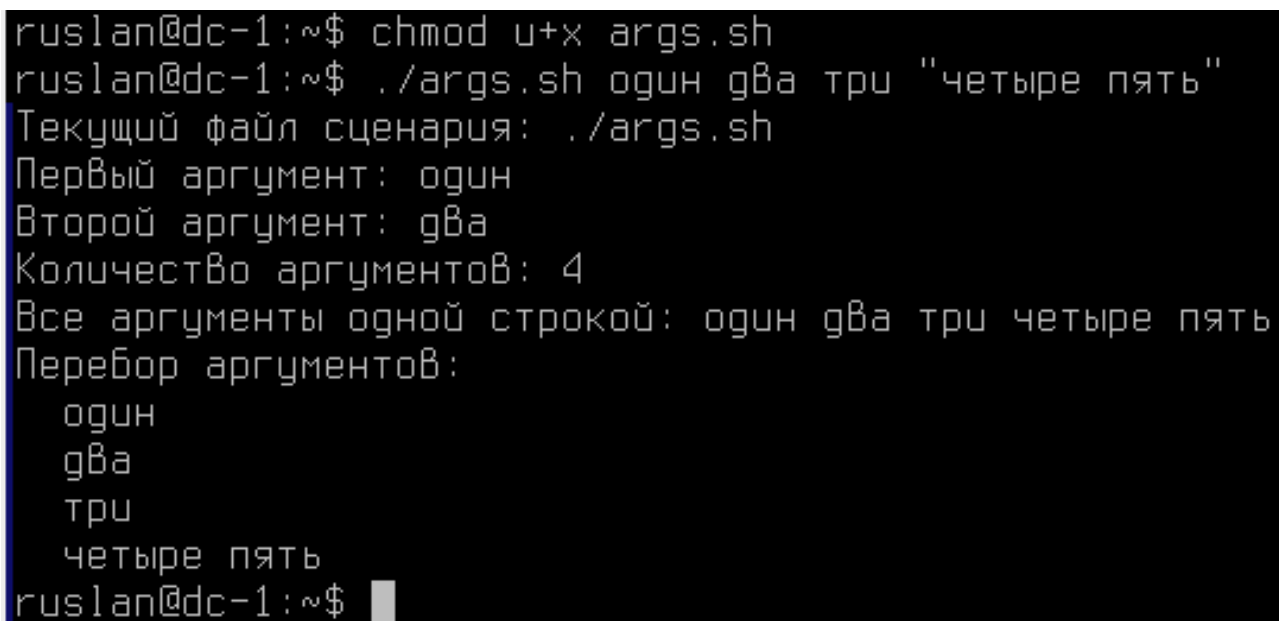
```
GNU nano 3.2 args.sh

#!/bin/bash
echo "Текущий файл сценария: $0"
echo "Первый аргумент: $1"
echo "Второй аргумент: ${2}"
echo "Количество аргументов: $#"
```

echo "Все аргументы одной строкой: \$*"
echo "Перебор аргументов:"
for arg in "\$@"; do
 echo " \$arg"
done

Рисунок 22 – Вывод всех аргументов одной строкой

6. Выведите в цикле «Все аргументы из переменной \$@».



```
ruslan@dc-1:~$ chmod u+x args.sh
ruslan@dc-1:~$ ./args.sh огун гва три "четыре пять"
Текущий файл сценария: ./args.sh
Первый аргумент: огун
Второй аргумент: гва
Количество аргументов: 4
Все аргументы одной строкой: огун гва три четыре пять
Перебор аргументов:
    огун
    гва
    три
    четыре пять
ruslan@dc-1:~$
```

Рисунок 23 – Результат выполнения скрипта args.sh с переданными аргументами, включая цикл по \$@

Вопросы

1. **Какой командой можно посмотреть информацию о типе команды?**

Ответ: `type` (например, `type echo`). В тексте упоминается `type` для проверки, является ли команда встроенной или внешней (стр. 13).

2. **Какая нотация используется для глобальных переменных?**

Ответ: Имена глобальных переменных (переменных окружения) традиционно пишутся заглавными буквами, например `PATH`, `USER` (стр. 18).

3. **Какие кавычки позволяют подставить значение переменной?**

Ответ: Двойные кавычки `" "` позволяют подставлять значения переменных (стр. 15).

4. **Какие кавычки экранируют специальный символ доллара \$, не позволяя подставлять значения переменных?**

Ответ: Одинарные кавычки `' '` экранируют все символы, включая `$` (стр. 15).

5. **Какие конструкции языка позволяют повторить блок кода несколько раз в зависимости от условия?**

Ответ: Циклы: `for`, `while`, `until` (стр. 13).

6. **Какой конструкцией языка можно выполнить ветвление кода?**

Ответ: Условный оператор `if ... then ... else ... fi` (стр. 11).

7. **Как в языке Bash можно получить значение позиционных переменных?**

Ответ: С помощью `$1`, `$2`, ... `${10}` и т.д. (стр. 19).

8. **Какой командой можно очистить переменную?**

Ответ: `unset имя_переменной` (стр. 18).

9. **С помощью какой специальной переменной можно получить значения всех аргументов?**

Ответ: `$*` или `$@` (стр. 19).

